中山大學 SUN YAT-SEN UNIVERSITY

国家超级计算广州中心 NATIONAL SUPERCOMPUTER CENTER IN GUANGZHOU

并行程序设计与算法（实验）

7-MPI并行应用

吴迪、刘学正

中山大学 计算机学院

实验概要

实验内容

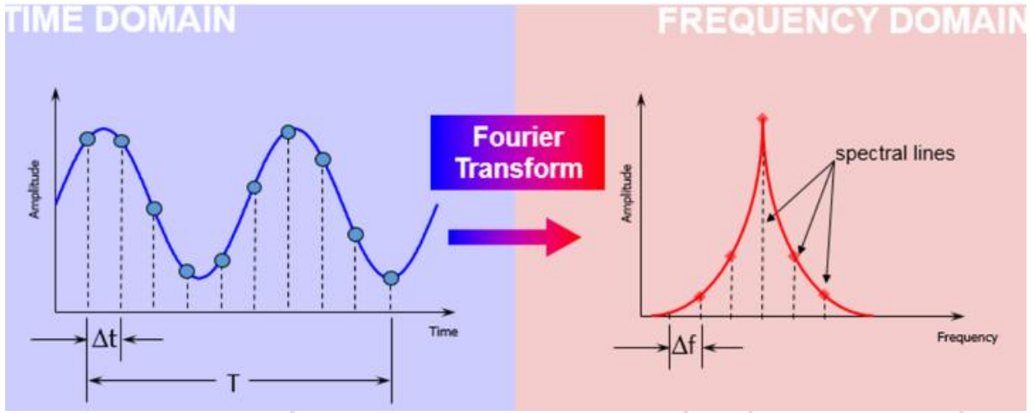
- 快速傅里叶变换分析与并行化改造
- MPI数据通信优化
- 程序性能与内存分析

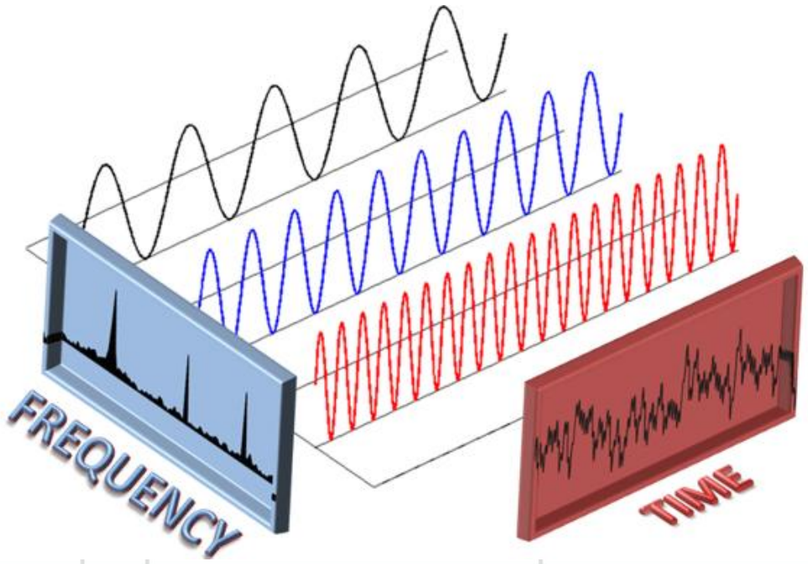
实验目的

- 验证并行FFT的加速效果与可扩展性
- 评估数据打包对通信性能的优化作用
- 分析并行规模对内存消耗的影响

背景介绍

- **离散傅里叶变换（英语Discrete Fourier Transform，缩写为DFT），**是信号处理中非常重要的一种数学工具，用于将一个**有限长度的任意离散序列**从**原域（时域/空域）**映射到**频域**
  - 输入：离散采样的有限长信号（如手机录音、传感器采样数据）
  - 输出：拆解出信号包含的各频率分量，以及对应分量的幅值、相位信息
- 它是**数字信号处理、图像处理、通信系统、雷达探测、医学成像**等众多领域的**核心基础算法**，支撑滤波、频谱分析、图像压缩、信号调制解调、目标探测、断层成像等关键技术实现





离散傅里叶变换

DFT形式化定义

设 *x(n)* 是一个长度为 *N* 的有限长序列，*x(n)* 的 *N* 点离散傅里叶变化（DFT）*X(k)* 同样是一个长度为 *N* 的序列，定义如下：

$$X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{kn} \quad 0 \le k \le N-1$$

$$W_N^{kn} = e^{-j\frac{2\pi kn}{N}} = \cos\left(\frac{2\pi kn}{N}\right) - j\sin\left(\frac{2\pi kn}{N}\right)$$ （旋转因子）

周期性：$W_N^{k+mN} = W_N^k, \quad m \in \mathbb{Z}$

尺度缩放性质：$W_{2N}^{2k} = W_N^k$

半周期对称性：$W_N^{k+\frac{N}{2}} = -W_N^k$

离散傅里叶变换

- 8点DFT计算示例：

偶数索引的列可利用 $W_8^{2k} = W_4^k$ 性质，简化为 4 点 DFT 形式

$$\begin{pmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \\ X[4] \\ X[5] \\ X[6] \\ X[7] \end{pmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & W_8 & W_8^2 & W_8^3 & W_8^4 & W_8^5 & W_8^6 & W_8^7 \\ 1 & W_8^2 & W_8^4 & W_8^6 & W_8^8 & W_8^{10} & W_8^{12} & W_8^{14} \\ 1 & W_8^3 & W_8^6 & W_8^9 & W_8^{12} & W_8^{15} & W_8^{18} & W_8^{21} \\ 1 & W_8^4 & W_8^8 & W_8^{12} & W_8^{16} & W_8^{20} & W_8^{24} & W_8^{28} \\ 1 & W_8^5 & W_8^{10} & W_8^{15} & W_8^{20} & W_8^{25} & W_8^{30} & W_8^{35} \\ 1 & W_8^6 & W_8^{12} & W_8^{18} & W_8^{24} & W_8^{30} & W_8^{36} & W_8^{42} \\ 1 & W_8^7 & W_8^{14} & W_8^{21} & W_8^{28} & W_8^{35} & W_8^{42} & W_8^{49} \end{bmatrix} \begin{pmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \\ x[4] \\ x[5] \\ x[6] \\ x[7] \end{pmatrix}$$

- **复数加法：** $N(N-1)$
- **复数乘法：** N^2

算法复杂度： $O(N^2)$

快速傅里叶变换

- **快速傅里叶变换(Fast Fourier Transform, FFT)是一种高效计算DFT的算法，它大大降低了计算复杂度**
  - 分治法策略：库利-图基算法(Cooley-Tukey algorithm)

$$F_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & W_4^2 & W_4^3 \\ 1 & W_4^2 & W_4^4 & W_4^6 \\ 1 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix}$$

4点DFT矩阵

$$D_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & W_8^1 & 0 & 0 \\ 0 & 0 & W_8^2 & 0 \\ 0 & 0 & 0 & W_8^3 \end{bmatrix}$$

旋转因子对角矩阵

$$\underbrace{\begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \\ \hline X[4] \\ X[5] \\ X[6] \\ X[7] \end{bmatrix}}_{\text{输出}} = \underbrace{\begin{bmatrix} F_4 & D_4F_4 \\ F_4 & -D_4F_4 \end{bmatrix}}_{\text{拆分后的块矩阵}} \underbrace{\begin{bmatrix} x[0] \\ x[2] \\ x[4] \\ x[6] \\ \hline x[1] \\ x[3] \\ x[5] \\ x[7] \end{bmatrix}}_{\text{奇偶重排后的输入}}$$

1个8点DFT转化为2个4点DFT，可反复拆分

快速傅里叶变换(按时间抽取)

- **分治法策略：把输入序列 *x(n)* 按 *n* 的奇偶性分为两组**

$$X(k)=\sum_{n=0}^{N-1}x(n)W_N^{kn},\quad k=0,1,\ldots,N-1$$

按时间下标奇偶拆分

偶数项：$n=2r$，记 $x_1(r)=x(2r)$

奇数项：$n=2r+1$，记 $x_2(r)=x(2r+1)$

$$\begin{aligned}X(k)&=\sum_{r=0}^{N/2-1}x(2r)W_N^{k\cdot 2r}+\sum_{r=0}^{N/2-1}x(2r+1)W_N^{k(2r+1)}\\&=\sum_{r=0}^{N/2-1}x_1(r)W_N^{2kr}+\sum_{r=0}^{N/2-1}x_2(r)W_N^{2kr}\cdot W_N^k\\&=\sum_{r=0}^{N/2-1}x_1(r)W_{N/2}^{kr}+W_N^k\sum_{r=0}^{N/2-1}x_2(r)W_{N/2}^{kr}\end{aligned}$$

$$X_1(k)=\sum_{r=0}^{N/2-1}x_1(r)W_{N/2}^{kr}\qquad X_2(k)=\sum_{r=0}^{N/2-1}x_2(r)W_{N/2}^{kr}$$

$$X(k)=X_1(k)+W_N^kX_2(k)\quad k=0,1,\ldots,\frac{N}{2}-1$$

快速傅里叶变换(按时间抽取)

- **分治法策略：把输入序列 *x(n)* 按 *n* 的奇偶性分为两组**

$$X\left(k+\frac{N}{2}\right)=\sum_{r=0}^{N/2-1}x_1(r)W_{N/2}^{(k+N/2)r}+W_N^{k+N/2}\sum_{r=0}^{N/2-1}x_2(r)W_{N/2}^{(k+N/2)r}$$

$$W_{N/2}^{(k+N/2)r}=W_{N/2}^{kr}\cdot W_{N/2}^{(N/2)r}=W_{N/2}^{kr}$$

$$\begin{aligned}X\left(k+\frac{N}{2}\right)&=X_1\left(k+\frac{N}{2}\right)+W_N^{k+N/2}X_2\left(k+\frac{N}{2}\right)\\&=X_1(k)+(-W_N^k)X_2(k)\\&=X_1(k)-W_N^kX_2(k)\end{aligned}$$

$$\begin{aligned}X(k)&=X_1(k)+W_N^kX_2(k)\\X\left(k+\frac{N}{2}\right)&=X_1(k)-W_N^kX_2(k)\end{aligned}\qquad k=0,1,\ldots,\frac{N}{2}-1$$

N点DFT——>2个N/2点DFT

快速傅里叶变换(按频率抽取)

- **分治法策略：把输出序列 X(*k*) 按 *k* 的奇偶性分为两组**

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn}, \quad k = 0, 1, \ldots, N-1$$

按时域前后半段拆分输入

$$X(k) = \sum_{n=0}^{N/2-1} x(n) W_N^{kn} + \sum_{n=N/2}^{N-1} x(n) W_N^{kn}$$

令 $n = m + \frac{N}{2}$ $\left(m = 0, 1, \ldots, \frac{N}{2} - 1 \right)$ $W_N^{k(m+N/2)} = W_N^{km} \cdot W_N^{kN/2}$

$$X(k) = \sum_{n=0}^{N/2-1} x(n) W_N^{kn} + W_N^{kN/2} \sum_{n=0}^{N/2-1} x\left(n + \frac{N}{2}\right) W_N^{kn}$$

快速傅里叶变换(按频率抽取)

- **分治法策略：把输出序列 X(*k*) 按 *k* 的奇偶性分为两组**

偶频率： $k=2r$ （$r=0,1,\ldots,\frac{N}{2}-1$）

$$W_N^{kN/2}=W_N^{2r\cdot N/2}=W_N^{rN}=e^{-j2\pi r}=1$$

$$\begin{aligned}X(2r)&=\sum_{n=0}^{N/2-1}\left[x(n)+x\left(n+\frac{N}{2}\right)\right]W_N^{2rn}\\&=\sum_{n=0}^{N/2-1}\left[x(n)+x\left(n+\frac{N}{2}\right)\right]W_{N/2}^{rn}\end{aligned}$$

$$g(n)=x(n)+x\left(n+\frac{N}{2}\right),\quad n=0,1,\ldots,\frac{N}{2}-1$$

奇频率： $k=2r+1$ （$r=0,1,\ldots,\frac{N}{2}-1$）

$$\begin{aligned}W_N^{kN/2}&=W_N^{(2r+1)N/2}=W_N^{rN+N/2}=W_N^{rN}\cdot W_N^{N/2}\\&=1\cdot e^{-j\pi}=-1\end{aligned}$$

$$\begin{aligned}X(2r+1)&=\sum_{n=0}^{N/2-1}\left[x(n)-x\left(n+\frac{N}{2}\right)\right]W_N^{n}\cdot W_N^{2rn}\\&=\sum_{n=0}^{N/2-1}\left\{\left[x(n)-x\left(n+\frac{N}{2}\right)\right]W_N^{n}\right\}W_{N/2}^{rn}\end{aligned}$$

$$h(n)=\left[x(n)-x\left(n+\frac{N}{2}\right)\right]\cdot W_N^n,\quad n=0,1,\ldots,\frac{N}{2}-1$$

$$\begin{aligned}X(2r)&=\mathrm{DFT}_{N/2}\{g(n)\}\\X(2r+1)&=\mathrm{DFT}_{N/2}\{h(n)\}\end{aligned}\qquad r=0,1,\ldots,\frac{N}{2}-1$$

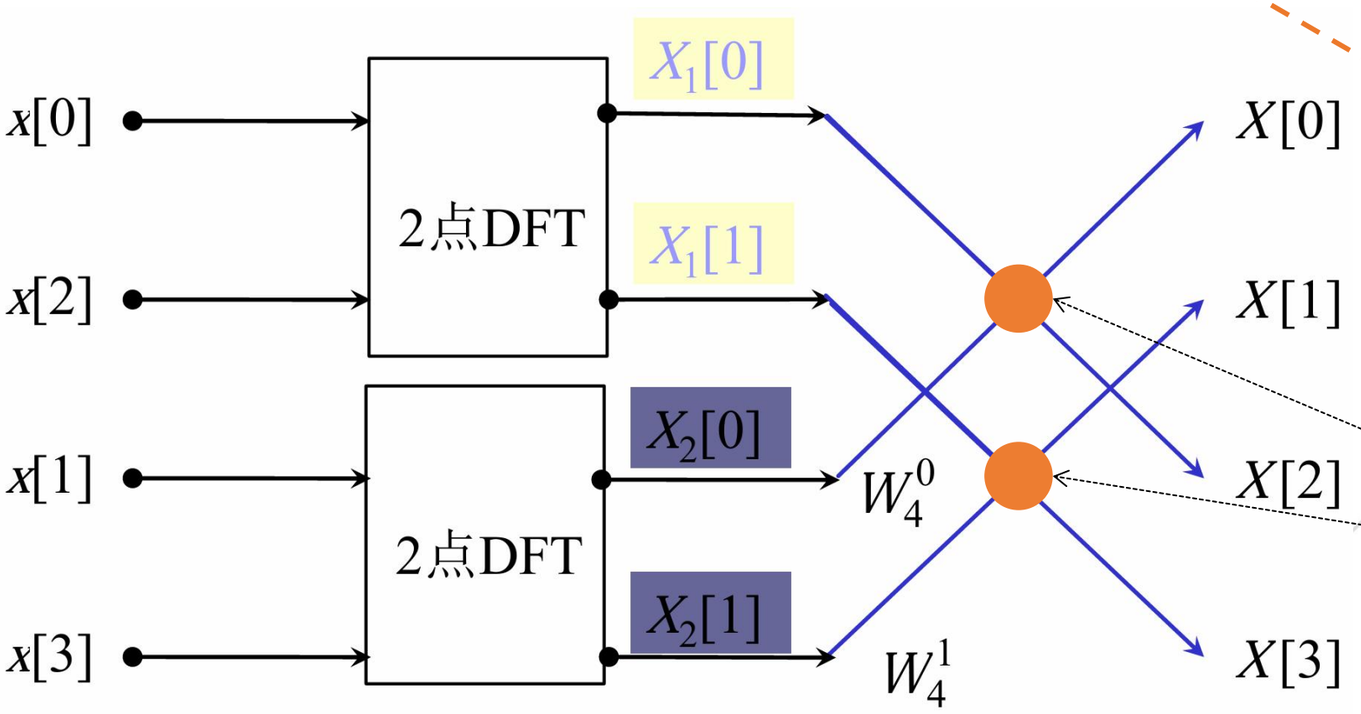
N点DFT——>2个N/2点DFT

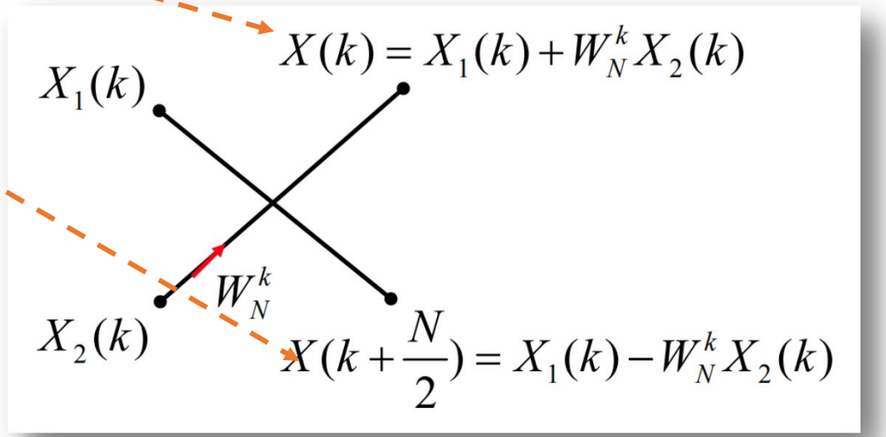
快速傅里叶变换示例

4点FFT计算示例（N=4）

$$X(k) = X_1(k) + W_4^k X_2(k), \quad k = 0,1$$

$$X(k+2) = X_1(k) - W_4^k X_2(k), \quad k = 0,1$$



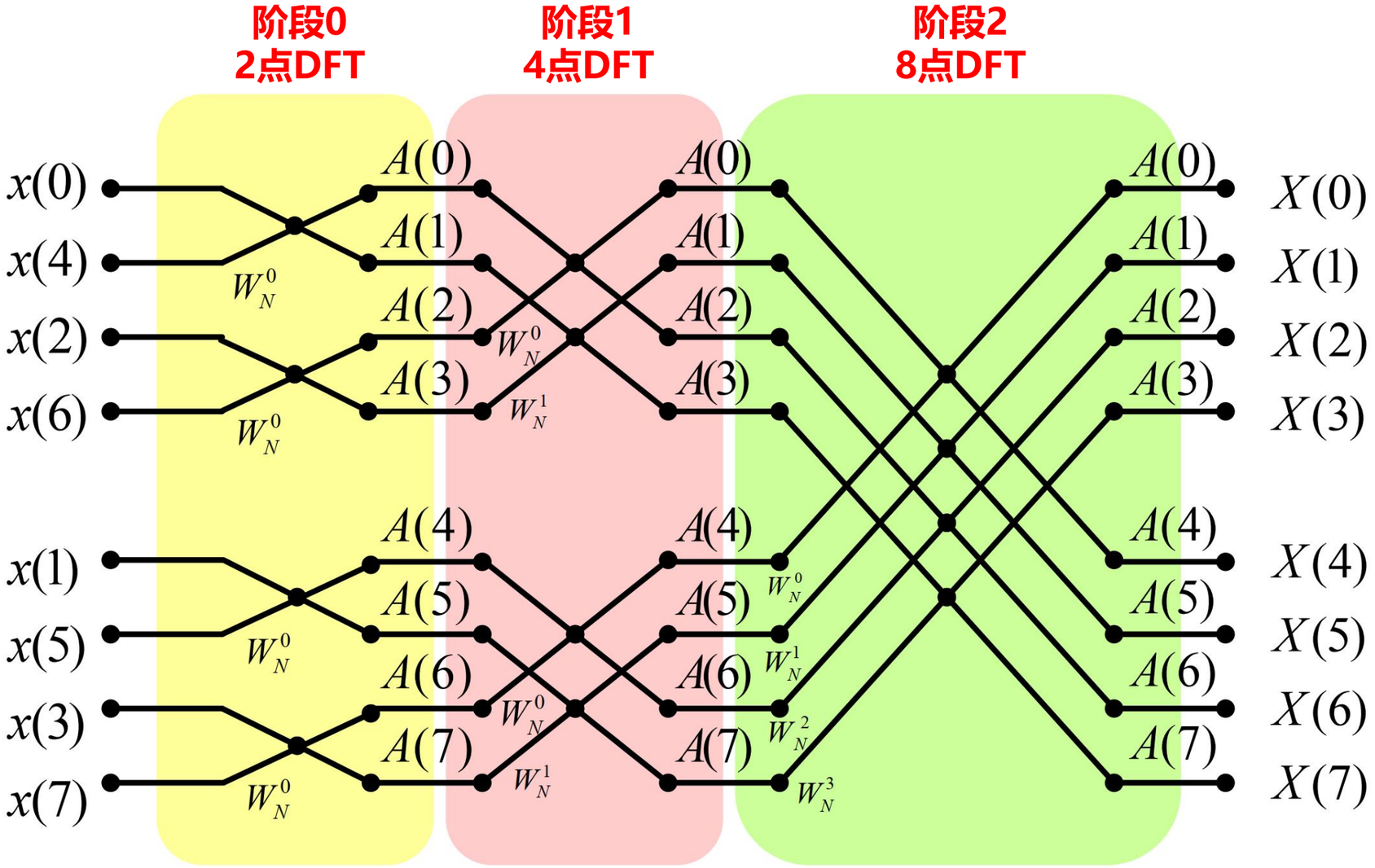


蝶形计算

1次乘法+两次加法

快速傅里叶变换示例

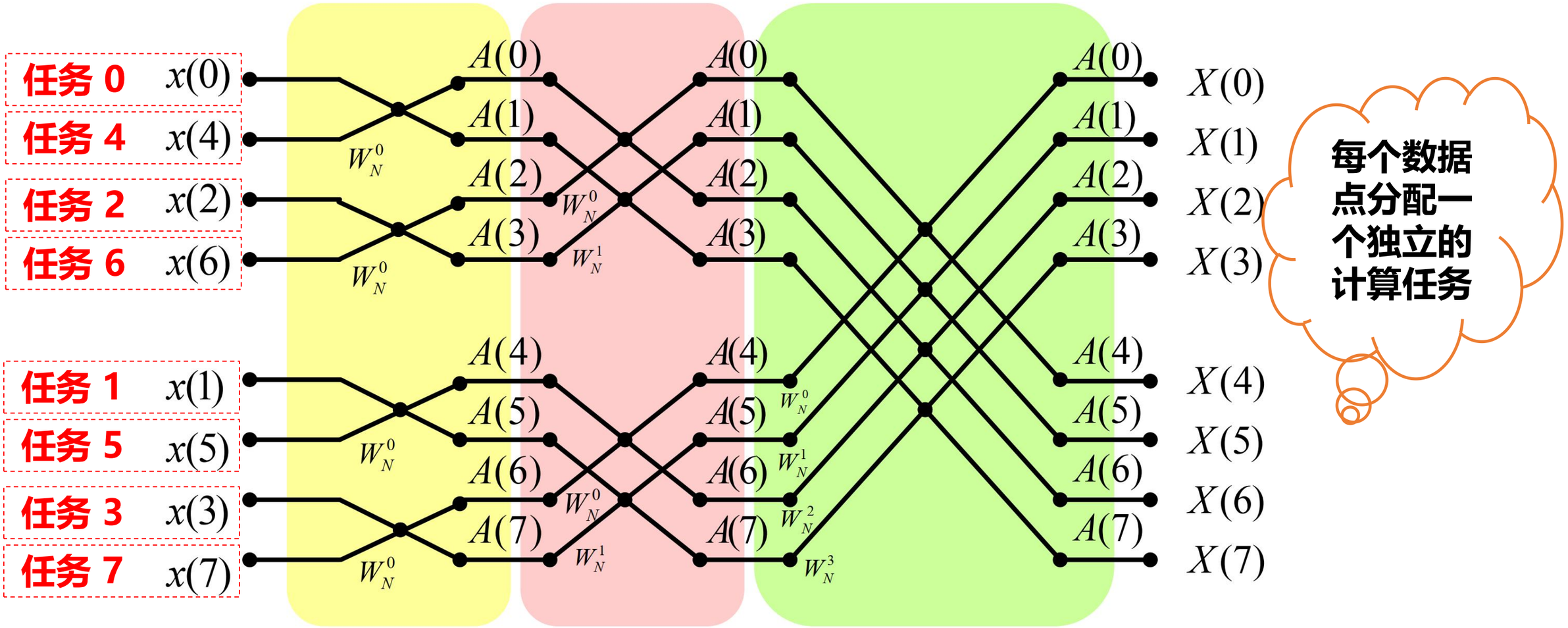
8点FFT计算示例（N=8）



- *log(N)*级蝶形运算
- 每1级有N/2个蝶形
  - 复数乘法：*N/2*
  - 复数加法：*N*
- *算法复杂度*
  - *O(Nlog N)*

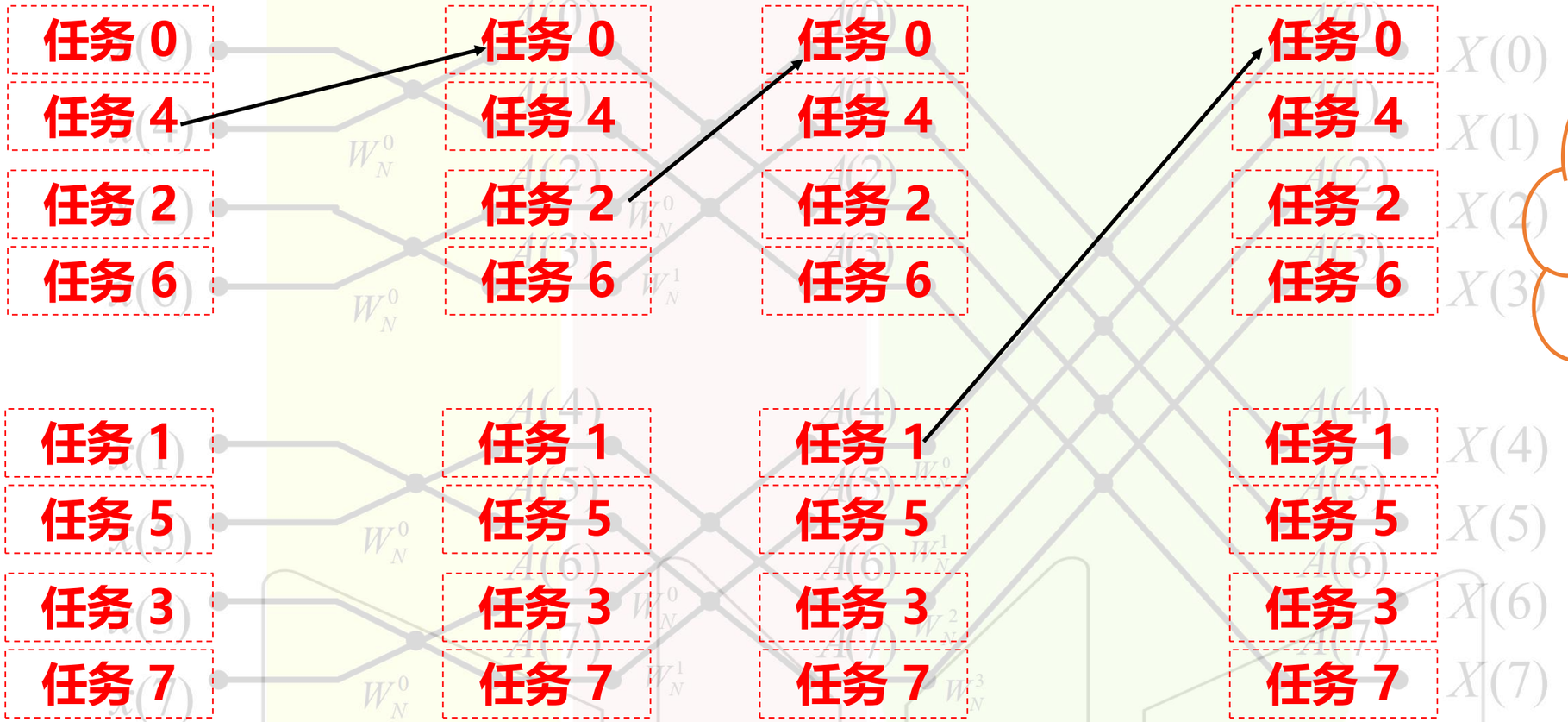
并行快速傅里叶变换

- 划分：按照序列长度 *N* 分为 *N* 个任务（*N=8*）



并行快速傅里叶变换

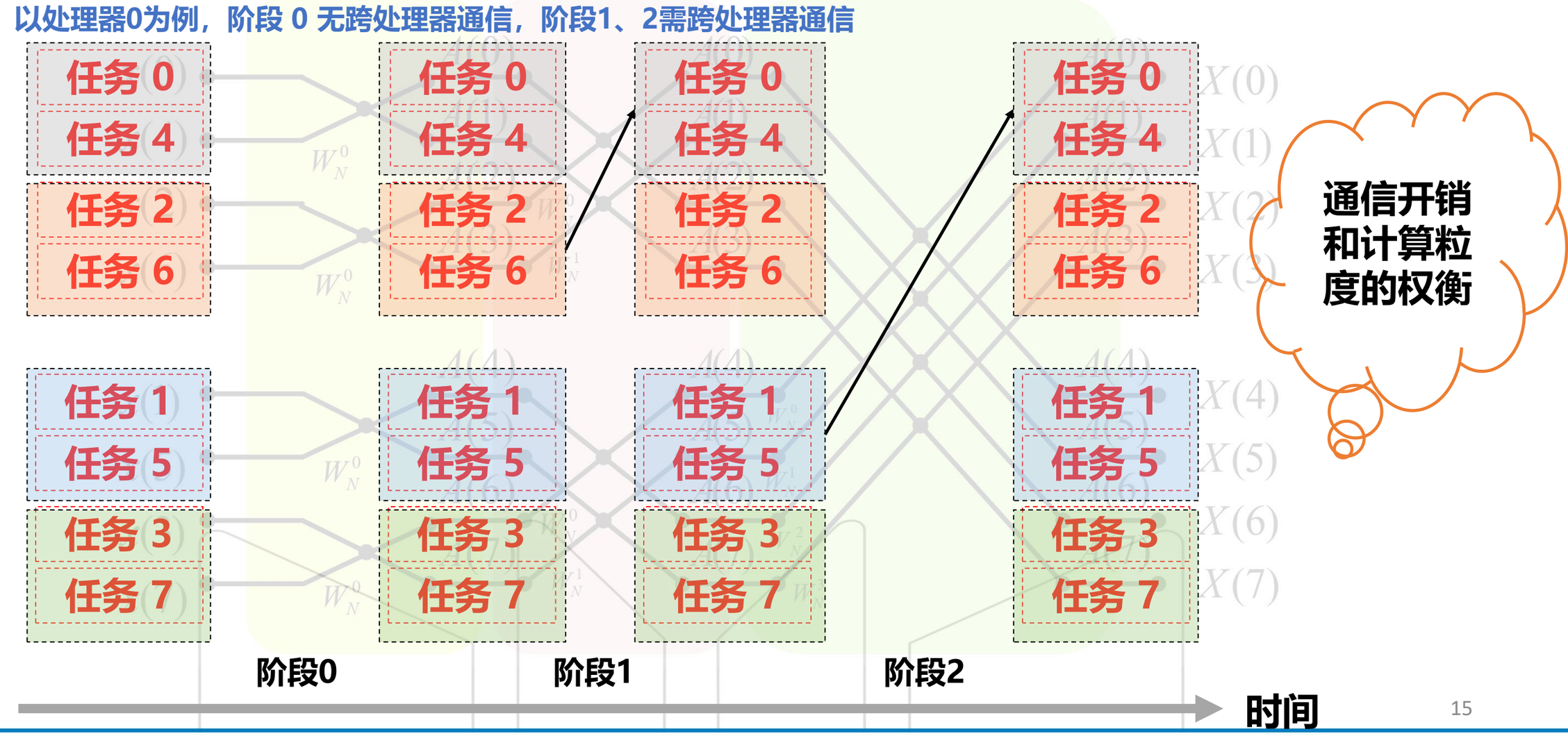
- 通信：以任务0为例，在不同计算阶段，需要依次与索引间隔为4，2，1的任务进行通信



通信模式呈现规律性 Hypercube

并行快速傅里叶变换

- 聚合和映射：将 N 个任务分配给 p 个处理器（$N=8$，$p=4$）

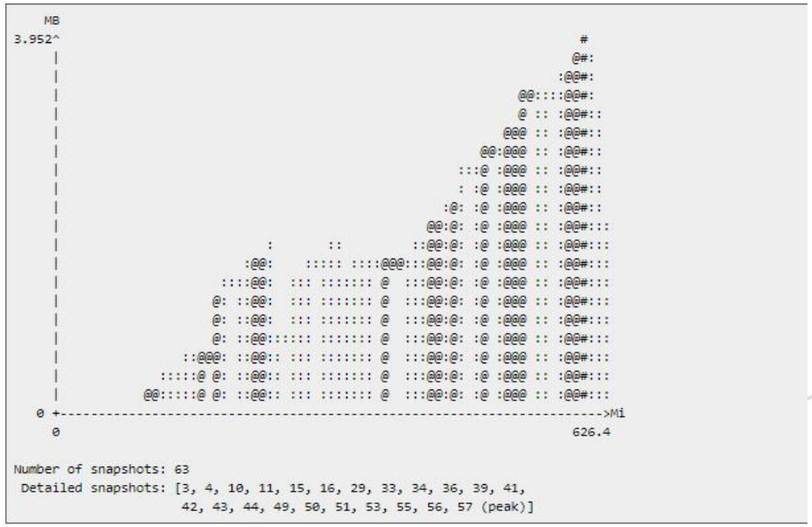


实验内容

- **使用MPI对快速傅里叶变换进行并行化**
  - 阅读参考文献中的串行傅里叶变换代码(fft_serial.cpp)
  - 使用MPI对其进行并行化（可能需要对原代码进行调整）
  - 通过数据打包优化消息传递效率
    - MPI_Pack/MPI-Unpack或MPI_Type_create_struct对数据重组
  - 性能分析
    - 分析不同并行规模（进程数）及问题规模（N）下的性能
    - 分析数据打包对于并行程序性能的影响
    - 使用valgrind massif工具集采集并分析程序的内存消耗

实验内容

- **使用MPI对快速傅里叶变换进行并行化**
  - 性能分析：使用valgrind massif工具集采集并分析内存消耗
    - Valgrind命令中增加--stacks=yes 参数采集程序运行栈内内存消耗
    - Valgrind massif输出日志（massif.out.pid）可由ms_print打印
      - x轴为程序运行时间，y轴为内存消耗量
    - 参考https://valgrind.org/docs/manual/ms-manual.html



Questions?

